🔄 Angular Velocity Calibration

1. Course Introduction

Verify the rotation angle calculation accuracy of the A1 Pro chassis wheel odometry (computed from the chassis MCU encoder and front wheel steering mechanism), and perform precise calibration of the angular velocity scale correction factor.

🎯 Course Objectives

1. Understand the angular velocity calibration principle and calculation formulas based on the Ackermann steering chassis
2. Master the calibration operation process using the `calibrate_angular_A1P` node combined with ground angle markers
3. Master the persistence method for writing the optimal angular velocity correction factor to the low-level driver parameter configuration file `a1_driver.yaml`

> **Important**
>
> The vehicle has been calibrated at the factory with baseline values. **No re-calibration is needed** for first-time use. You only need to refer to this tutorial for recalibration when long-term operation wear occurs, different tires are replaced, or when fixed proportional cumulative errors are detected in the robot's rotation at specific angles.

> **⚠ Warning**
>
> The A1 Pro uses an Ackermann front wheel servo steering structure and **does not support in-place rotation**. The calibration program will steer the wheels while moving forward at a given speed (default 0.15 m/s) to draw a circular path and accumulate rotation angles. **Please be sure to select a flat and open area of at least 2.5m × 2.5m for calibration.**

---

2. Preparation

2.1 Start the Chassis Comprehensive Driver

Open a terminal and run the following command to start all A1 Pro chassis drivers and the EKF fusion localization node:

```
ros2 launch yahboomcar_bringup yahboomcar_bringup_A1_launch.py
```

```
jetson@yahboom:~$ ros2 launch yahboomcar_bringup yahboomcar_bringup_A1_launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-08-28-17-40-33-437836-yahboom-77906
[INFO] [launch]: Default logging verbosity is set to INFO
--------------------robot_type = A1--------------------
[INFO] [joint_state_publisher-1]: process started with pid [77942]
[INFO] [robot_state_publisher-2]: process started with pid [77944]
[INFO] [Ackman_driver_A1P-3]: process started with pid [77946]
[INFO] [base_node_A1-4]: process started with pid [77948]
[INFO] [imu_attitude_converter-5]: process started with pid [77950]
[INFO] [ekf_node-6]: process started with pid [77954]
[base_node_A1-4] [INFO] [1787910033.723024238] [base_node]: A1 odometry: linear_scale_x=1.000000 angular_scale_z=1.000000 wheelbase_
compat=0.250 max_dt=0.500
[robot_state_publisher-2] [INFO] [1787910033.732425135] [robot_state_publisher]: got segment Gripping
[robot_state_publisher-2] [INFO] [1787910033.734185567] [robot_state_publisher]: got segment arm1
[robot_state_publisher-2] [INFO] [1787910033.735261213] [robot_state_publisher]: got segment arm2
[robot_state_publisher-2] [INFO] [1787910033.736229847] [robot_state_publisher]: got segment arm3
[robot_state_publisher-2] [INFO] [1787910033.737131600] [robot_state_publisher]: got segment arm4
[robot_state_publisher-2] [INFO] [1787910033.737959526] [robot_state_publisher]: got segment arm5
[robot_state_publisher-2] [INFO] [1787910033.738848063] [robot_state_publisher]: got segment arm_base_Link
[robot_state_publisher-2] [INFO] [1787910033.739773432] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1787910033.741424165] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1787910033.742278300] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1787910033.743267223] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-2] [INFO] [1787910033.744055021] [robot_state_publisher]: got segment laser_Link
[robot_state_publisher-2] [INFO] [1787910033.744752032] [robot_state_publisher]: got segment lbw_Link
[robot_state_publisher-2] [INFO] [1787910033.744782657] [robot_state_publisher]: got segment lfr_Link
[robot_state_publisher-2] [INFO] [1787910033.744795841] [robot_state_publisher]: got segment lfw_Link
[robot_state_publisher-2] [INFO] [1787910033.744806849] [robot_state_publisher]: got segment llink1
[robot_state_publisher-2] [INFO] [1787910033.744817186] [robot_state_publisher]: got segment llink2
[robot_state_publisher-2] [INFO] [1787910033.744826626] [robot_state_publisher]: got segment llink3
[robot_state_publisher-2] [INFO] [1787910033.744836098] [robot_state_publisher]: got segment rbw_Link
[robot_state_publisher-2] [INFO] [1787910033.744847971] [robot_state_publisher]: got segment rfr_Link
[robot_state_publisher-2] [INFO] [1787910033.744857059] [robot_state_publisher]: got segment rfw_Link
[robot_state_publisher-2] [INFO] [1787910033.744865347] [robot_state_publisher]: got segment rlink1
[robot_state_publisher-2] [INFO] [1787910033.744874467] [robot_state_publisher]: got segment rlink2
[robot_state_publisher-2] [INFO] [1787910033.744882947] [robot_state_publisher]: got segment rlink3
[joint_state_publisher-1] [INFO] [1787910034.486409578] [joint_state_publisher]: Waiting for robot_description to be published on th
e robot_description topic...
[imu_attitude_converter-5] [INFO] [1787910034.625107672] [imu_display_tf]: IMU attitude converter: /imu/data_raw -> /imu/data, publi
sh_tf=False
[Ackman_driver_A1P-3] [INFO] [1787910035.285285998] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s
```

2.2 Prepare Ground Initial Markers

- Use tape to lay out a crosshair or a clear reference direction line on open ground;
- Place the robot on the marked line with the vehicle's longitudinal center axis strictly parallel to the reference line, and record the initial heading direction.

---

3. Calibration Process

3.1 Start the Angular Velocity Calibration Program

Run the calibration test node in a new terminal:

```
ros2 run yahboomcar_bringup calibrate_angular_A1P
```

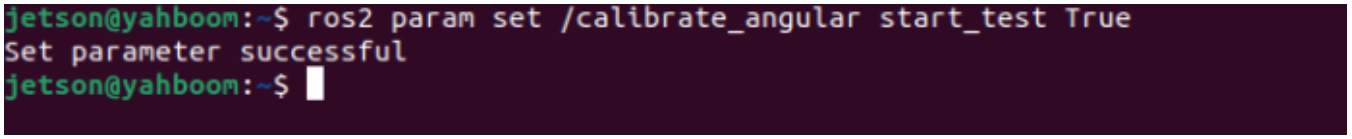
```
jetson@yahboom:~$ ros2 run yahboomcar_bringup calibrate_angular_A1
[INFO] [1787910505.892072613] [calibrate_angular]: Waiting for TF transfor
ms to be ready...
[INFO] [1787910507.895509787] [calibrate_angular]: ========================
============================
[INFO] [1787910507.896194224] [calibrate_angular]: Angular calibration nod
e started
[INFO] [1787910507.896757857] [calibrate_angular]: Start testing with comm
and:
[INFO] [1787910507.897300146] [calibrate_angular]:   ros2 param set /calib
rate_angular start_test True
[INFO] [1787910507.897840674] [calibrate_angular]: ========================
============================
```

The calibration node will listen to the real-time rotation quaternion from `odom -> base_footprint` in the TF tree and convert it to yaw angle.

3.2 Trigger Rotation Test

In another terminal, trigger the test via ROS 2 dynamic parameters:

```
ros2 param set /calibrate_angular start_test True
```



The default test target angle is **360°** (one full circle). The robot will turn counterclockwise in a circular path while moving forward with a preset angular velocity (default 1.0 rad/s). When the odometry accumulated rotation angle reaches 360°, the robot will automatically decelerate and stop.

3.3 Compare with Actual Physical Angle on Ground

After the robot comes to a complete stop, observe the angle between the robot's current physical heading and the initial reference direction line on the ground:

- **Ideal state**: The vehicle has rotated exactly one full circle (360°), and the heading direction is perfectly parallel and aligned with the ground reference line;
- **Under-rotated (actual rotation < 360°)**: The heading stopped before reaching the target position;
- **Over-rotated (actual rotation > 360°)**: The vehicle stopped after rotating past the reference line.

3.4 Calculate and Dynamically Adjust the Correction Factor

The calibration program and chassis driver accumulate rotation angle by integrating angular velocity. The angle increment is **multiplied by the correction factor** (i.e., **determined rotation angle = raw angular displacement × correction factor**):

- **Writing a factor greater than 1.0**: Angular velocity feedback is amplified, and the odometry considers the vehicle to be rotating faster, **reaching 360° earlier and stopping earlier** (the actual physical rotation angle of the robot **becomes smaller**);
- **Writing a factor less than 1.0**: Angular velocity feedback is reduced, and the odometry considers the vehicle to be rotating slower, **delaying the stop** (the actual physical rotation angle of the robot **becomes larger**).

Therefore, based on the actual physical rotation angle on the ground and the expected target angle (360°), the new correction factor calculation formula is:

```
New Correction Factor = Current Factor × (Actual Physical Rotation Angle / Target Rotation Angle)
```

Parameter Adjustment Examples and Calculations:

- **Scenario 1: Over-rotated (actual rotation angle > 360°)**
  Target rotation set to 360.0°, current factor is 1.0, robot rotated past the reference line and stopped at 370.0° (over-rotated by 10°):

  ```
  New Factor = 1.0 × (370.0 / 360.0) ≈ 1.0278 (increase the factor)
  ```

  After increasing the factor from 1.0 to 1.0278, the odometry accumulation becomes faster, the robot will brake and stop earlier, and the actual rotation angle will decrease accordingly, stopping precisely on the 360° reference line.

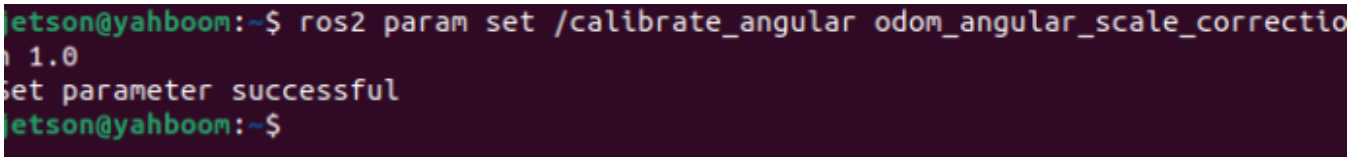
- **Scenario 2: Under-rotated (actual rotation angle < 360°)**
  Target rotation set to 360.0°, current factor is 1.0, robot stopped early at 350.0° before reaching the target (still 10° short):

  ```
  New Factor = 1.0 × (350.0 / 360.0) ≈ 0.9722 (decrease the factor)
  ```

  After decreasing the factor from 1.0 to 0.9722, the odometry accumulation becomes slower, and the robot needs to rotate a bit more to reach 360°, thus compensating for the angle and precisely aligning with the reference line.

Dynamically set the new factor via command line (example of increasing to 1.0278):

```
ros2 param set /calibrate_angular odom_angular_scale_correction 1.0278
```



> **Tip**
>
> Core rule:
>
> - If the actual physical rotation angle on the ground is **greater than** 360° (over-rotated) **increase** the correction factor;
> - If the actual physical rotation angle on the ground is **less than** 360° (under-rotated) **decrease** the correction factor.

3.5 Multiple Verifications and Convergence

Move the robot back to the starting point and align with the reference line. Run `ros2 param set /calibrate_angular start_test True` again. Observe whether the robot's heading precisely aligns with the initial reference line when it stops with the new factor applied. Repeat until the visual physical angle error is within 2°.

---

4. Persist the Correction Factor to Driver Configuration

After calibration, to ensure the low-level driver automatically applies the settings on every boot or launch, write the optimal calibrated factor to the `driver_node` parameter configuration file:

Parameter file path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/a1_driver.yaml
```

Open `a1_driver.yaml`, find the `odom_angular_scale_z` configuration item and modify it to the final calibrated value (e.g., 1.0278):

```yaml
driver_node:
  ros__parameters:
    # ...
    # Chassis angular velocity odometry scale correction factor (new calibrated value)
    odom_linear_scale_x: 1.0
    odom_angular_scale_z: 1.0278
    # ...
```

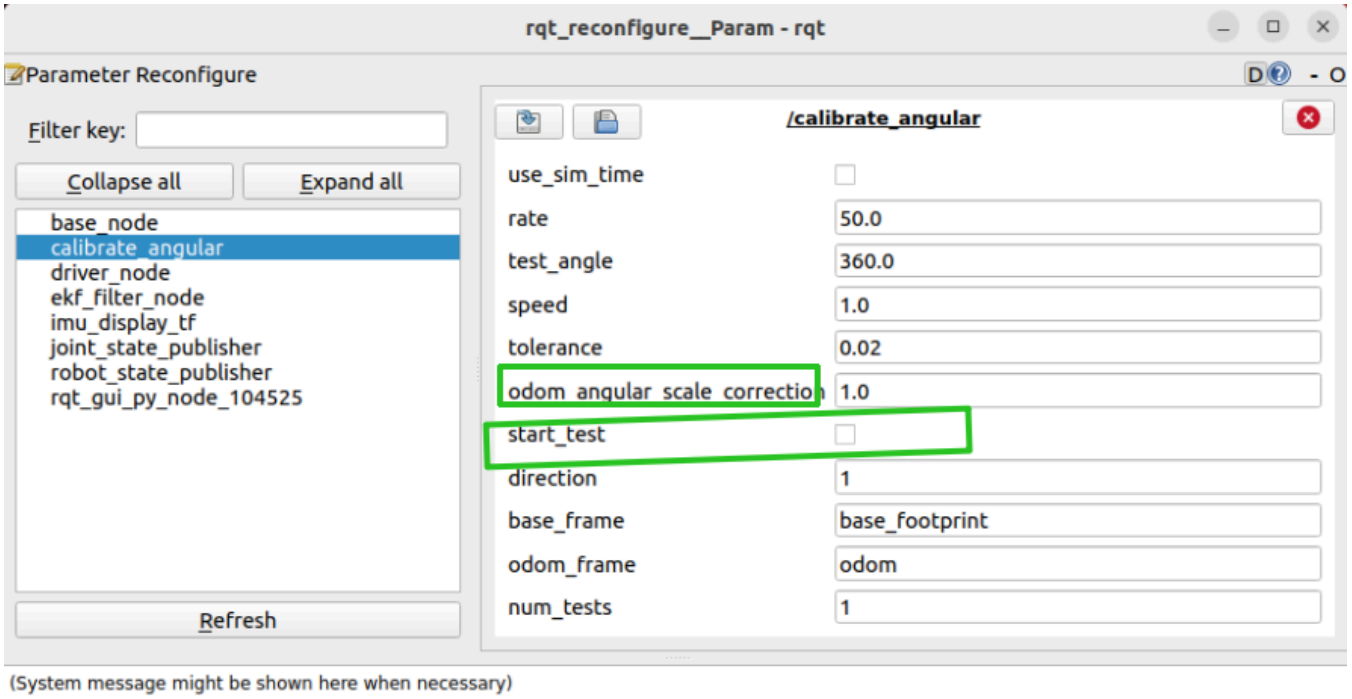


> **Note**
>
> After modifying `a1_driver.yaml`, it is recommended to recompile the package to synchronize the configuration file in the installation path:
>
> ```
> cd ~/yahboomcar_ros2_ws/yahboomcar_ws
> colcon build --packages-select yahboomcar_bringup
> source install/setup.bash
> ```

5. Using rqt_reconfigure for GUI Parameter Tuning

In addition to the command line, you can also use `rqt_reconfigure` provided by ROS 2 for visual real-time parameter tuning and test triggering:

```
ros2 run rqt_reconfigure rqt_reconfigure
```



> **Important**
>
> **How to handle when no nodes are displayed in the interface**:
> If `/calibrate_angular` node is **not shown** in the left panel after opening `rqt_reconfigure`, this is due to timing issues between node startup and tool loading where it was not automatically refreshed. Simply click the **Refresh button (Refresh)** in the upper left corner of the rqt interface to rescan and display `/calibrate_angular`.

5.1 Calibration Node Parameter Description

The `/calibrate_angular` node panel contains the following key parameters:

Parameter Name	Default Value	Physical Meaning and Description
`start_test`	`False`	Test trigger switch (check to start test, automatically unchecks when rotation is complete)
`odom_angular_scale_correction`	`1.0`	Angular velocity scale correction factor for real-time testing within the calibration node (>1.0 stops earlier with smaller rotation angle, <1.0 stops later with larger rotation angle)
`test_angle`	`360.0`	Target test rotation angle (degrees)
`speed`	`1.0`	Test rotation angular velocity (rad/s)

Parameter Name	Default Value	Physical Meaning and Description
`linear_speed`	`0.15`	Ackermann chassis forward linear velocity during turning (m/s)
`direction`	`1`	Rotation direction: `1` counterclockwise (left turn), `-1` clockwise (right turn)
`tolerance`	`0.02`	Stop angle tolerance (rad, approximately 1.15°)
`num_tests`	`1`	Number of consecutive test rounds (when set to multiple, automatically alternates between forward and reverse rotation)
`rate`	`50.0`	TF query and control main loop frequency (Hz)
`base_frame`	`base_footprint`	Robot chassis projection base coordinate frame name
`odom_frame`	`odom`	Odometry global reference coordinate frame name

6. Core Source Code and Algorithm Principle Analysis

Source code path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/calibrate_angular_A1P.py
```

6.1 Angle Integration and Closed-Loop Control Logic

```python
def on_timer(self):
    if self.is_testing:
        current_angle = self.get_odom_angle() # Get current yaw angle from TF
        if current_angle is None:
            return

        # 1. Normalize angle increment and accumulate with correction factor
        delta = self.normalize_angle(current_angle - self.last_angle)
        self.turn_angle += delta * odom_scale
        self.last_angle = current_angle

        target = test_angle * self.direction
        error = target - self.turn_angle

        # 2. Ackermann chassis linear velocity + angular velocity coordinated control
        if abs(error) > tolerance:
            move_cmd = Twist()
```

```
            move_cmd.linear.x = abs(linear_speed)       # Maintain forward linear velocity
            move_cmd.angular.z = copysign(speed, error) # Steering angular velocity
            self.cmd_vel_pub.publish(move_cmd)
        else:
            # 3. Target rotation angle reached, publish stop
            self.cmd_vel_pub.publish(Twist())
            self.is_testing = False
```

6.2 Source Code and Driver Activation Mechanism Analysis

- **Why increasing the correction factor actually results in smaller rotation**:
  In the calibration program, the accumulated rotation angle condition is `turn_angle = sum(delta) * odom_scale`.
  When the robot stops, the actual rotation angle of the odometry is `target / odom_scale`.
  Therefore, when `odom_scale > 1.0` (increased factor), the denominator becomes larger for the actual required rotation angle, and the vehicle only needs to rotate a smaller angle for the program to determine the target is reached and stop; conversely, when `odom_scale < 1.0` (decreased factor), the vehicle must rotate a larger angle to reach the target.
- **Low-level driver activation mechanism**:
  In `Ackman_driver_A1P.py`, the driver node scales the angular velocity feedback with `angular_z * odom_angular_scale_z` in real time, and participates in the trapezoidal dead-reckoning `_integrate_and_publish_odom`. After persisting the angular velocity calibration factor to `a1_driver.yaml`, it can significantly improve the heading accumulation drift issue during Nav2 path tracking and rotation obstacle avoidance.